Approximating a Sinusoid as A Polynomial

```
clc; clear; close all;

addpath('BuildTables');
```

Problem

Consider the function:

$$x(t) = A\sin(\omega\, t + \phi)$$

We wish to find explicit formulas for the coefficients of a polynomial in terms of *t*.

That is we want a formula for S_k:

$$x(t) = \sum_k S_k(\omega, \phi) t^k$$

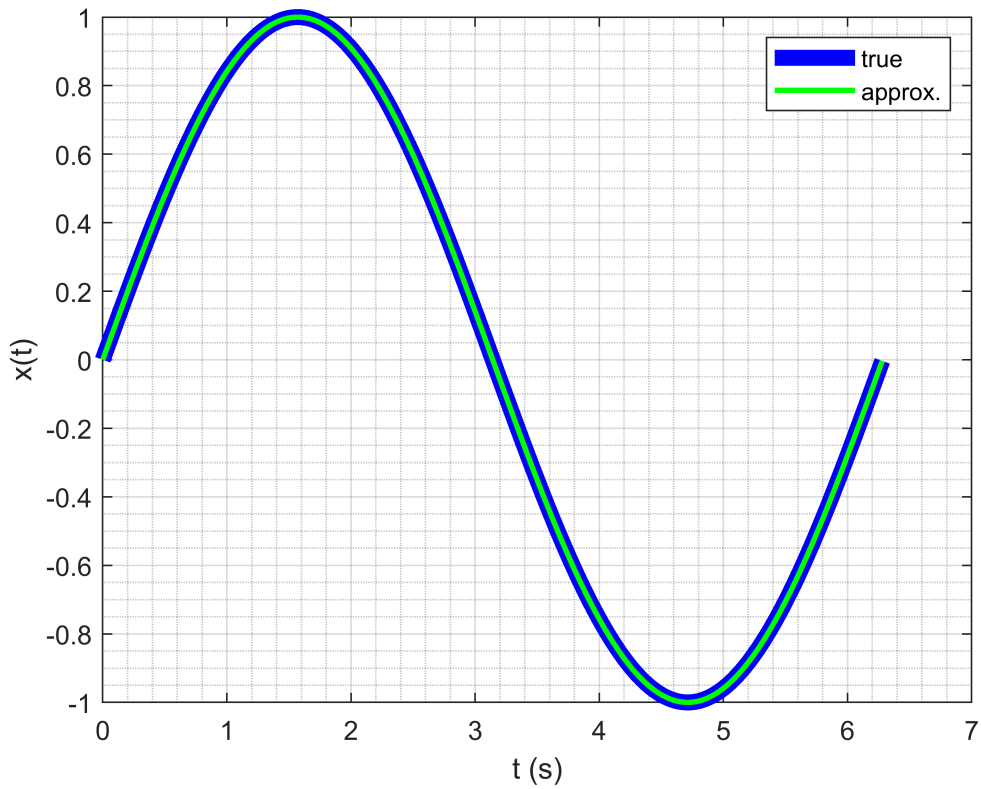
Simple Taylor Expansion

We start with a simple Taylor expansion:

Lemma 1:

$$A\sin(\omega\, t + \phi) = A\sum_{k=0}^{\infty} \frac{(-1)^k (\omega\, t + \phi)^{2k+1}}{(2k+1)!}$$

```
TestLemma1();
```



Application of the Binomial Theorem

Now we apply binomial expansion to the $(\omega\, t + \phi)^{2k+1}$ term arising in Lemma 1:

$$(\omega\, t + \phi)^{2k+1} = \sum_{j=0}^{2k+1} C_j^{2k+1} \phi^{\,2k+1-j} \omega^{\,j} t^j$$

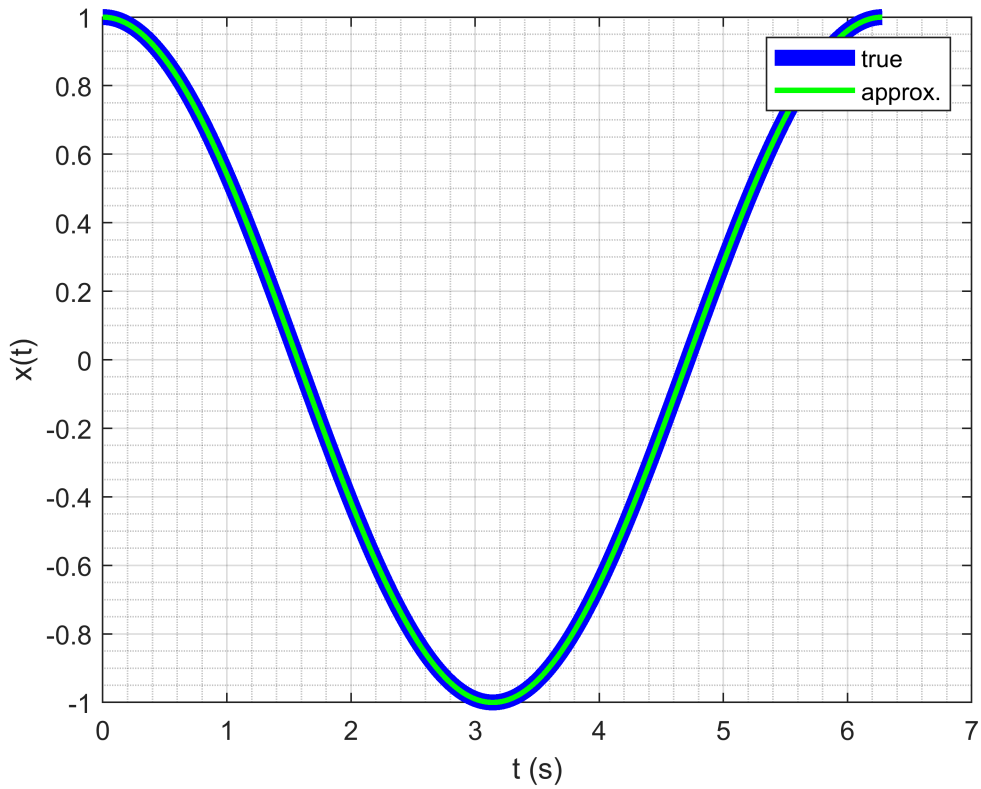
$$x(t) = A \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} \sum_{j=0}^{2k+1} C_j^{2k+1} \phi^{\,2k+1-j} \omega^{\,j} t^j$$

$$x(t) = A \sum_{k=0}^{\infty} \sum_{j=0}^{2k+1} \frac{(-1)^k}{(2k+1)!} C_j^{2k+1} \phi^{\,2k+1-j} \omega^{\,j} t^j$$

Lemma 2:

$$A \sin(\omega\, t + \phi) = A \sum_{k=0}^{\infty} \sum_{j=0}^{2k+1} \frac{(-1)^k}{(2k+1)!} C_j^{2k+1} \phi^{\,2k+1-j} \omega^{\,j} t^j$$

```
TestLemma2();
```



Rearranging the Summation

Now we wish to rearrange the summation from Lemma 2 into a sum in terms of powers of *t*.

The sum from Lemma 2 was:

$$A\sin(\omega t+\phi) = A\sum_{k=0}^{\infty}\sum_{j=0}^{2k+1}\frac{(-1)^k}{(2k+1)!}C_j^{2k+1}\phi^{\,2k+1-j}\omega^{\,j}t^j$$

An approximate version can be considered to make the rearrangement of the summation more intuitive:

$$A\sin(\omega t+\phi) \approx A\sum_{k=0}^{K}\sum_{j=0}^{2k+1}\frac{(-1)^k}{(2k+1)!}C_j^{2k+1}\phi^{\,2k+1-j}\omega^{\,j}t^j$$

To rearrange the sum, note that the k and j indicies inside the summation loop form a pattern. For example, consider the (k,j) tuples formed inside the inner summation loop for K = 2

(0,0) (0,1)

(1,0) (1,1) (1,2) (1,3)

(2,0) (2,1) (2,2) (2,3) (2,4) (2,5)

This makes evident the fact that the summation can be broken into an even j and an odd j part:

Even j:

(0,0)

(1,0) (1,2)

(2,0) (2,2) (2,4)

Odd j:

(0,1)

(1,1) (1,3)

(2,1) (2,3) (2,5)

The full summation can be computed by summing the intermediate results from both these triangular arrays. Note that because the j index represents powers of t, the even summation part representes the cosine component of the wave and the odd summation part represents the sine component.

It is trivial to write loops over these two triangular arrays (see TestLemma3() for code).

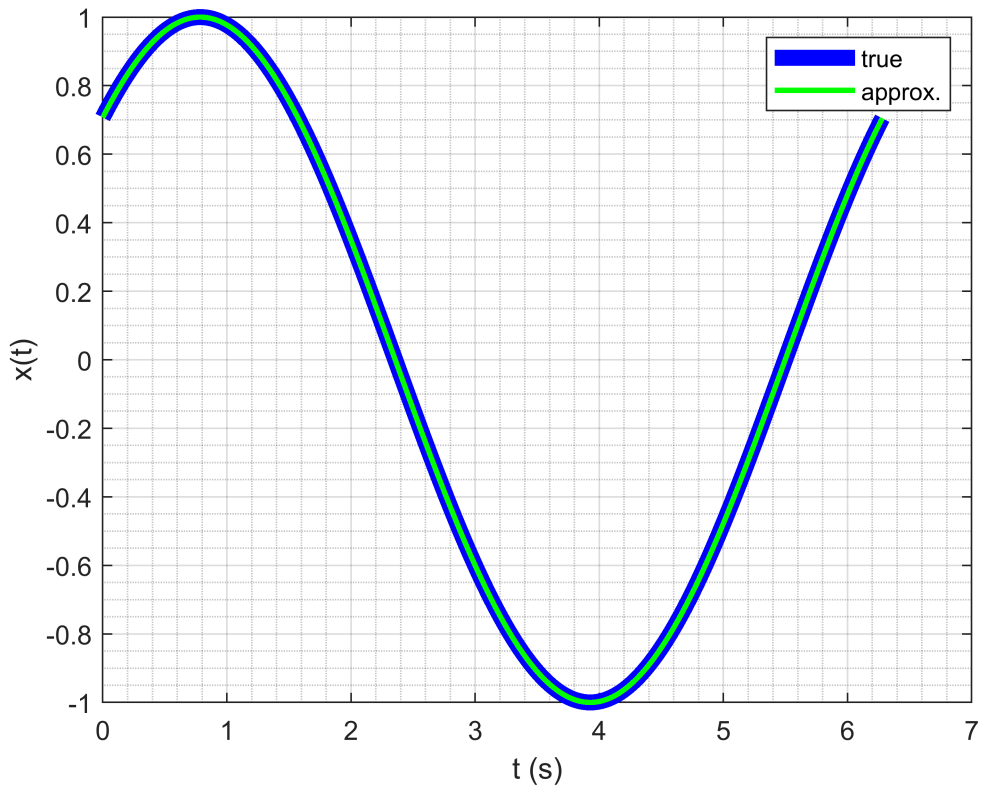
Therefore:

Lemma 3:

$$A\sin(\omega t+\phi)=\sum_{j=0}^{\infty}\sum_{k=j}^{\infty}\mu\,(2j,k)t^{2j}+\sum_{j=0}^{\infty}\sum_{k=j}^{\infty}\mu\,(2j+1,k)t^{2j+1}$$

$$\mu(j,k)=A\frac{(-1)^k}{(2k+1)!}C_j^{2k+1}\phi^{\,2k+1-j}\omega^{\,j}$$

```
TestLemma3();
```



Manipulating Lemma 3

Now we wish to manipulate Lemma 3 into a convienient matrix form which allows easily changing amplitude, frequency, or phase:

$$A\sin(\omega t+\phi) = \sum_{j=0}^{\infty}\sum_{k=j}^{\infty}\mu\,(2j,k)t^{2j} + \sum_{j=0}^{\infty}\sum_{k=j}^{\infty}\mu\,(2j+1,k)t^{2j+1}$$

$$\mu(j,k) = A\frac{(-1)^k}{(2k+1)!}C_j^{2k+1}\phi^{\,2k+1-j}\omega^{\,j}$$

Remove the ϕ and ω terms from the definition of μ:

$$A\sin(\omega t+\phi) = A\sum_{j=0}^{\infty}t^{2j}\omega^{\,2j}\sum_{k=j}^{\infty}\mu\,(2j,k)\Phi\,(2j,k) + A\sum_{j=0}^{\infty}t^{2j+1}\omega^{\,2j+1}\sum_{k=j}^{\infty}\mu\,(2j+1,k)\Phi\,(2j+1,k)$$

$$\mu(j,k) = \frac{(-1)^k}{(2k+1)!}C_j^{2k+1}$$

$$\Phi\,(j,k) = \phi^{\,2k+1-j}$$

Now define new functions, which give the elements of four upper triangular matricies:

$$\mu_E(j,k) = \begin{cases} \dfrac{(-1)^k}{(2k+1)!} C_{2j}^{2k+1} & k \geq j \\ 0 & o.w. \end{cases}$$

$$\mu_O(j,k) = \begin{cases} \dfrac{(-1)^k}{(2k+1)!} C_{2j+1}^{2k+1} & k \geq j \\ 0 & o.w. \end{cases}$$

$$\Phi_E(j,k,\phi) = \begin{cases} \phi^{\,2k+1-2j} & k \geq j \\ 0 & o.w. \end{cases}$$

$$\Phi_O(j,k,\phi) = \begin{cases} \phi^{\,2k+1-(2j+1)} = \phi^{\,2k-2j} & k \geq j \\ 0 & o.w. \end{cases}$$

Now note that the above sum can be expressed convieniently in matrix notation:

Lemma 4:

$$A\sin(\omega t + \phi) = \mathrm{T}_E \mathrm{K}_E + \mathrm{T}_O \mathrm{K}_O$$

If only one time point *t* is to be evaluated:

$$\mathrm{T}_E = \begin{bmatrix} t^0 & t^2 & t^4 & \cdots & t^{2K} \end{bmatrix} \in M^{1\times K}$$

$$\mathrm{T}_O = \begin{bmatrix} t^1 & t^3 & t^5 & \cdots & t^{2K+1} \end{bmatrix} \in M^{1\times K}$$

If *N* time points $t_0 \cdots t_N$ are to be evaluated:

$$\mathrm{T}_E = \begin{bmatrix} t_0^0 & t_0^2 & t_0^4 & \cdots & t_0^{2K} \\ t_1^0 & t_1^2 & t_1^4 & \cdots & t_1^{2K} \\ t_2^0 & t_2^2 & t_2^4 & \cdots & t_2^{2K} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ t_N^0 & t_N^2 & t_N^4 & \cdots & t_N^{2K} \end{bmatrix} \in M^{N\times K}$$

$$\mathrm{T}_O = \begin{bmatrix} t_0^1 & t_0^3 & t_0^5 & \cdots & t_0^{2K+1} \\ t_1^1 & t_1^3 & t_1^5 & \cdots & t_1^{2K+1} \\ t_2^1 & t_2^3 & t_2^5 & \cdots & t_2^{2K+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ t_N^1 & t_N^3 & t_N^5 & \cdots & t_N^{2K+1} \end{bmatrix} \in M^{N\times K}$$

$$\mathrm{K}_E = \mathrm{diag}((\mathrm{M}_E \circ \Phi_E)\Omega_E)$$

$$\mathrm{K}_O = \mathrm{diag}((\mathrm{M}_O \circ \Phi_O)\Omega_O)$$

$$\mathrm{M}_E = [\mu_E(j,k)] \in M^{K\times K}$$

$\mathrm{M}_O = [\mu_O(j,k)] \in M^{K\times K}$

$\Phi_E = [\Phi_E(j,k,\phi\ )] \in M^{K\times K}$

$\Phi_O = [\Phi_O(j,k,\phi\ )] \in M^{K\times K}$

$$\omega_E = \begin{bmatrix} \omega^0 & 0 & \cdots & 0 \\ \omega^0 & \omega^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \omega^0 & \omega^2 & \cdots & \omega^{2K} \end{bmatrix}$$

$$\omega_O = \begin{bmatrix} \omega^1 & 0 & \cdots & 0 \\ \omega^1 & \omega^3 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \omega^1 & \omega^3 & \cdots & \omega^{2K+1} \end{bmatrix}$$

As an optimization, note that

$\mathrm{K}_E = \mathrm{diag}((\mathrm{M}_E \circ \Phi_E)\Omega_E)$

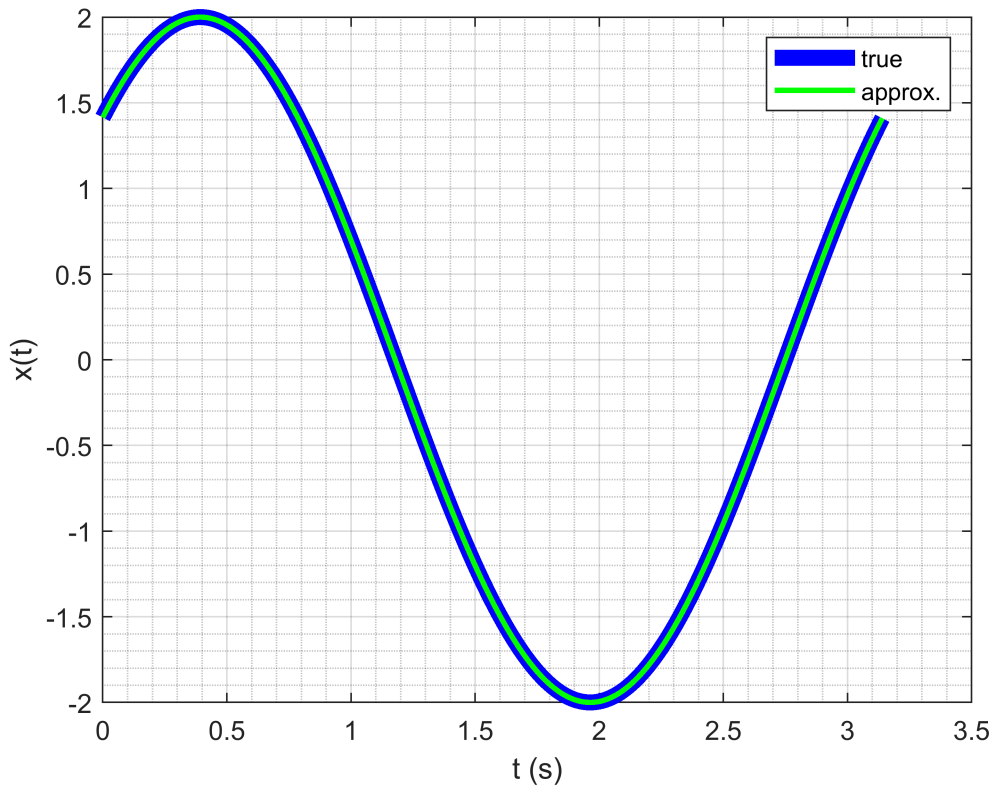
$\mathrm{K}_O = \mathrm{diag}((\mathrm{M}_O \circ \Phi_O)\Omega_O)$

Can be more efficiently computed as:

$\mathrm{K}_E = \mathrm{sum}\left(\mathrm{M}_E \circ \Phi_E \circ \Omega_E^T, 2\right)$

$\mathrm{K}_O = \mathrm{sum}\left(\mathrm{M}_O \circ \Phi_O \circ \Omega_O^T, 2\right)$

```
TestLemma4();
```

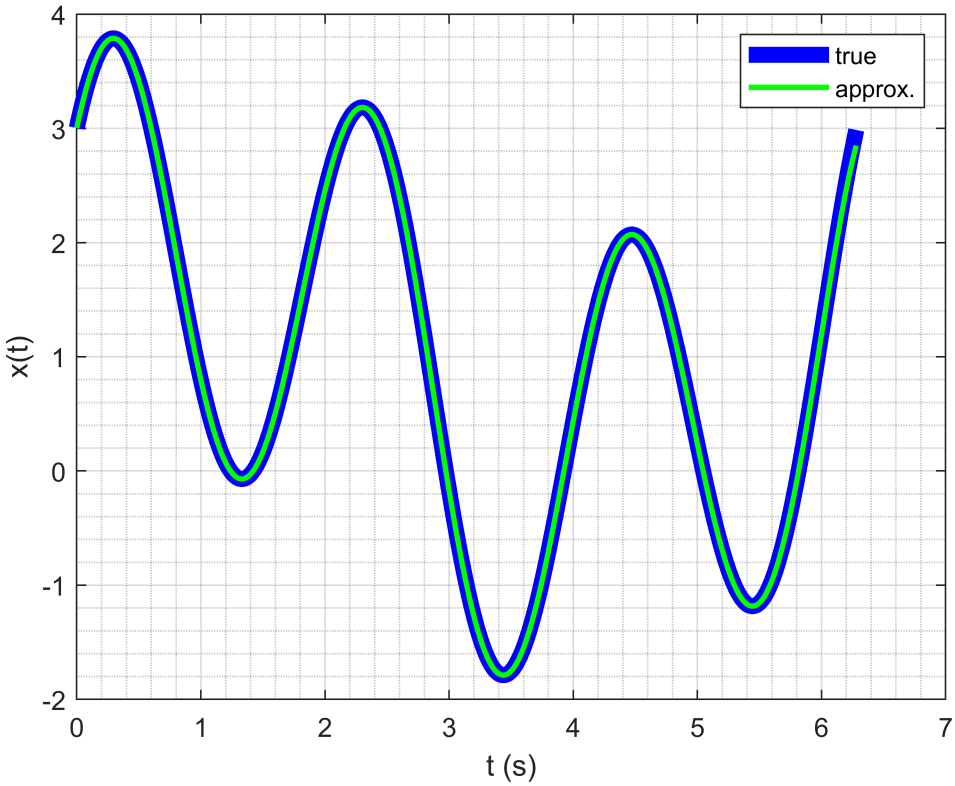


Biased Multisine Expansion

Lemma 4 can be used to evaluate a multisine, defined by a vector of Amplitudes, A, a vector of angular frequencies, ω, and a vector of phaseshifts, ϕ.

```
TestKappaSine
```

```
kappa_multisine_even_matrix = 26×1
    3.0020
   -6.6579
    4.7975
   -1.4327
    0.2301
   -0.0230
    0.0016
   -0.0001
    0.0000
   -0.0000
      ⋮
```


true
approx.
x(t)
t (s)
4
3
2
1
0
-1
-2
0
1
2
3
4
5
6
7

```
function [kappa_multisine_even_matrix, kappa_multisine_odd_matrix] = ...
   BuildKappaMultisine(A, omega, phi, bias, K)

kappa_multisine_even_matrix = zeros(K+1, 1);
kappa_multisine_odd_matrix = zeros(K+1, 1);

for i = 1:length(A)

    [kappa_sine_even_matrix, kappa_sine_odd_matrix] = ...
         BuildKappaSine(A(i), omega(i), phi(i), K);

    kappa_multisine_even_matrix = ...
        kappa_multisine_even_matrix + kappa_sine_even_matrix;

    kappa_multisine_odd_matrix = ...
        kappa_multisine_odd_matrix + kappa_sine_odd_matrix;

end

kappa_multisine_even_matrix(1) = ...
    kappa_multisine_even_matrix(1) + bias

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function [kappa_sine_even_matrix, kappa_sine_odd_matrix] = ...
   BuildKappaSine(A, omega, phi, K)

% build the even and odd mu tables for the sinusoid
mu_sine_even_matrix = BuildMuSineEvenMatrix(K);
mu_sine_odd_matrix = BuildMuSineOddMatrix(K);

% build the even and odd Phi tables for the sinusoid
phi_sine_even_matrix = BuildPhiSineEvenMatrix(K, phi);
phi_sine_odd_matrix = BuildPhiSineOddMatrix(K, phi);

% build the even and odd omega vectors
omega_sine_even_matrix = BuildOmegaSineEvenMatrix(K, omega);
omega_sine_odd_matrix = BuildOmegaSineOddMatrix(K, omega);

% build the kappa vectors accorting to Lemma 4
hadamard_even = mu_sine_even_matrix.*phi_sine_even_matrix;
hadamard_odd = mu_sine_odd_matrix.*phi_sine_odd_matrix;

% A more efficient way of computing diag(x*y) is to use sum(x.*y', 2)
kappa_sine_even_matrix = A*sum(hadamard_even.*omega_sine_even_matrix', 2);
kappa_sine_odd_matrix = A*sum(hadamard_odd.*omega_sine_odd_matrix', 2);

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function mu_sine_even_matrix = BuildMuSineEvenMatrix(K)

mu_sine_even_matrix = zeros(K,K);

for j = 0:K
    for k = 0:K
        % mu_sine_even_table(k, j) in C++
        mu_sine_even_matrix(j+1, k+1) = ComputeMuSineEven(j, k);
    end
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function mu_sine_odd_matrix = BuildMuSineOddMatrix(K)

mu_sine_odd_matrix = zeros(K,K);

for j = 0:K
    for k = 0:K
        % mu_sine_odd_table(k, j) in C++
        mu_sine_odd_matrix(j+1, k+1) = ComputeMuSineOdd(j, k);
    end
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function omega_sine_even_matrix = BuildOmegaSineEvenMatrix(K, omega)

omega_sine_even_matrix = zeros(K, K);

% Build a lower triangular matrix where each column is an even power of k
for j = 0:K
    for k = 0:K
        if k <= j
            % omega_sine_odd_vector(j, k) in C++
            omega_sine_even_matrix(j+1, k+1) = omega.^(2*k);
        end
    end
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function omega_sine_odd_matrix = BuildOmegaSineOddMatrix(K, omega)

omega_sine_odd_matrix = zeros(K, K);

% Build a lower triangular matrix where each column is an odd power of k
for j = 0:K
    for k = 0:K
        if k <= j
            % omega_sine_odd_matrix(j, k) in C++
            omega_sine_odd_matrix(j+1, k+1) = omega.^(2*k+1);
        end
    end
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function phi_sine_even_matrix = BuildPhiSineEvenMatrix(K, phi)

phi_sine_even_matrix = zeros(K,K);

for j = 0:K
    for k = 0:K
        % phi_sine_even_table(j, k) in C++
        phi_sine_even_matrix(j+1, k+1) = ComputePhiSineEven(j, k, phi);
    end
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function phi_sine_odd_matrix = BuildPhiSineOddMatrix(K, phi)

phi_sine_odd_matrix = zeros(K,K);

for j = 0:K
    for k = 0:K
        % phi_sine_odd_table(j, k) in C++
        phi_sine_odd_matrix(j+1, k+1) = ComputePhiSineOdd(j, k, phi);
    end
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
% @breif Each column in the matrix is a power of t, each row is an element
% in the batch of all t data points
function tau_even_matrix = BuildTauEvenMatrix(K, t)

t = t';

N = length(t);

tau_even_matrix = zeros(N, K);

for j = 0:K
    tau_even_matrix(:,j+1) = t.^(2*j); % tau_even_matrix(:,j) in C++
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
% @breif Each column in the matrix is a power of t, each row is an element
% in the batch of all t data points
function tau_odd_matrix = BuildTauOddMatrix(K, t)

t = t';

N = length(t);

tau_odd_matrix = zeros(N, K);

for j = 0:K
    tau_odd_matrix(:,j+1) = t.^(2*j+1); % tau_odd_matrix(:,j) in C++
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function [x] = EvaluateKappaMultisine( ...
    kappa_sine_even_matrix, kappa_sine_odd_matrix, t, K)

% build the tau matricies
tau_even_matrix = BuildTauEvenMatrix(K, t);
tau_odd_matrix = BuildTauOddMatrix(K, t);

% Apply Lemma 4
even_part = tau_even_matrix*kappa_sine_even_matrix;
odd_part = tau_odd_matrix*kappa_sine_odd_matrix;

x = odd_part + even_part;

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function  mu_sine_even = ComputeMuSineEven(j, k)

if k >= j
    quotient = (-1)^k/factorial(2*k+1);
    binomial_coef = nchoosek(2*k+1, 2*j);
    mu_sine_even = quotient*binomial_coef;
else
    mu_sine_even = 0;
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function  mu_sine_odd = ComputeMuSineOdd(j, k)

if k >= j
    quotient = (-1)^k/factorial(2*k+1);
    binomial_coef = nchoosek(2*k+1, 2*j+1);
    mu_sine_odd = quotient*binomial_coef;
else
    mu_sine_odd = 0;
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function phi_sine_even = ComputePhiSineEven(j, k, phi)

if k >= j
    phi_sine_even = phi^(2*k+1-2*j);
else
    phi_sine_even = 0;
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function phi_sine_odd = ComputePhiSineOdd(j, k, phi)

if k >= j
    phi_sine_odd = phi^(2*k - 2*j);
else
    phi_sine_odd = 0;
end

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function [x] = EvaluateMultisine(A, omega, phi, bias, t)

x = zeros(size(t)) + bias;

for i = 1:length(A)
    x = x + A(i).*sin(omega(i).*t + phi(i));
end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function TestKappaSine

% Sinusoid parameters
A = [1 2];
omega = [1 3];
phi = [pi/5 pi/4];
bias = 1;
t = 0:0.01:2*pi/omega(1);

% Approximation parameters
K = 25;

% Get truth to test against
x_true = EvaluateMultisine(A, omega, phi, bias, t);

% Build the Kappa approximation
[kappa_sine_even_matrix, kappa_sine_odd_matrix] = ...
   BuildKappaMultisine(A, omega, phi, bias, K);

% Evaluate the Kappa approximation
x_approx = EvaluateKappaMultisine( ...
    kappa_sine_even_matrix, kappa_sine_odd_matrix, t, K);

figure;
plot(t, x_true, 'b', 'linewidth', 6);
hold on;
plot(t, x_approx, 'g', 'linewidth', 2);
xlabel('t (s)');
ylabel('x(t)');
legend('true', 'approx.');
grid on;
grid minor;

end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function TestLemma1()

omega = 1;
phi = 0;
A = 1;
t = 0:0.01:2*pi/omega;

% truth
x_true = A.*sin(omega.*t + phi);

% approx
K = 20;
sum = 0;

for k = 0:K
    term = (-1)^k*(omega*t+phi).^(2*k+1)/factorial(2*k+1);
    sum = sum + term;
end

x_approx = A.*sum;

figure;
plot(t, x_true, 'b', 'linewidth', 6);
hold on;
plot(t, x_approx, 'g', 'linewidth', 2);
xlabel('t (s)');
ylabel('x(t)');
legend('true', 'approx.');
grid on;
grid minor;
end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)

```
function TestLemma2()

omega = 1;
phi = pi/2;
A = 1;
t = 0:0.01:2*pi/omega;

% truth
x_true = A.*sin(omega.*t + phi);

% approx
K = 20;
outer_sum = 0;

for k = 0:K

    inner_sum = 0;

    for j = 0:2*k+1
        quotient = (-1)^k/factorial(2*k+1);
        binomial_coef = nchoosek(2*k+1, j);
        term = quotient*binomial_coef*phi^(2*k+1-j)*omega^j.*t.^j;
        inner_sum = inner_sum + term;
    end

    outer_sum = outer_sum + inner_sum;
end

x_approx = A.*outer_sum;

figure;
plot(t, x_true, 'b', 'linewidth', 6);
hold on;
plot(t, x_approx, 'g', 'linewidth', 2);
xlabel('t (s)');
ylabel('x(t)');
legend('true', 'approx.');
grid on;
grid minor;
end
```

You created this PDF from an application that is not licensed to print to novaPDF printer (http://www.novapdf.com)